INFORME TÉCNICO DE INGENIERÍA INVERSA Y DISEÑO LÓGICO FORMAL (v3)

Proyecto: Biometría y Monitorización Medioambiental IoT (Sprint 0)
Rol: Ingeniero de Software Principal & Arquitecto de Sistemas
Ámbito: Arquitectura Multiplataforma — Nodo IoT Firmware C++/Arduino ↔ Gateway Móvil Android/Java ↔ Backend REST & UX Web PHP/JS

Documento consolidado
Especificación lógica, arquitectura, auditoría de defectos y propuesta de refactorización descritas en la versión v3 suministrada.

Contenido

- Fase 1 — Diagrama de diseño lógico y modelo formal.
- Fase 2 — Comentarios y documentación de código.
- Fase 3 — Auditoría de errores y vulnerabilidades.
- Fase 4 — Mejora y refactorización.

FASE 1. DIAGRAMA DE DISEÑO LÓGICO

Modelo formal agnóstico del sistema completo, concebido como plantilla para representación manuscrita y como contrato lógico entre capas.

1.1 Modelo de tipos abstractos, compuestos y agregaciones

Tipos primitivos y abstractos

- $\mathbb{N}$ (N): números naturales / sin signo, $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$. Corresponde a uint8_t, uint16_t, uint32_t, size_t.
- $\mathbb{Z}$ (Z): enteros con signo, $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$. Corresponde a int, int8_t, int16_t, int32_t, long y byte (Java).
- $\mathbb{R}$ (R): números reales en coma flotante. Corresponde a float y double.
- $\mathbb{B}$ (B): booleanos algebraicos, $\mathbb{B} = \{\text{true}, \text{false}\}$.
- Text: cadenas de caracteres alfanuméricos contiguos (std::string, char*, String).

Tipos compuestos y enumeraciones

- MedicionesID = {CO2 = 11, TEMPERATURA = 12, RUIDO = 13}.
- ModoSeguridadBLE = {ABIERTO, SIN_ACCESO}.

Agregaciones

- UUID128 = $[N]_{16}$.
- TramaRAW = $[Z]_{30}$.
- RespuestaREST = (codigo : Z, cuerpo : Text).
- RespuestaLogin = (resultado : B, usuario : Text, error : Text).
- RespuestaSaludo = (nombre : Text, saludo : Text, error : Text).

1.2 Convención de lectura de los bloques ASCII

- Límite interno: atributos privados encapsulados.
- Pared vertical izquierda (|): métodos públicos. Flecha → hacia el método si recibe parámetros; flecha ← desde el método si retorna datos.
- Separador ||: aislamiento visual entre métodos consecutivos.
- Pared vertical derecha: flechas de estado interno: → mutador / non-const; ← consultor / const / read-only; --x estático.

1.3 Subsistema Nodo Sensor IoT — Firmware C++ / nRF52840

CLASE: LED

Atributos privados	Métodos públicos / contratos
• numeroLED : Z • encendido : B	• LED() — recibe numero : Z → • encender() → • apagar() → • alternar() → • brillar(tiempo : Z) →

CLASE: PuertoSerie

Atributos privados	Métodos públicos / contratos
• baudios : Z	• PuertoSerie(baudios : Z) → • esperarDisponible() ← B • escribir(mensaje : Text) →

CLASE: Medidor

Atributos privados	Métodos públicos / contratos

• (sin estado interno mutable en Sprint 0)	• Medidor() → • iniciarMedidor() → • medirCO2() ← valorCO2 : Z • medirTemperatura() ← valorTemp : Z

CLASE: Caracteristica (sub-entidad de ServicioEnEmisora)

Atributos privados	Métodos públicos / contratos
• uuidCaracteristica : $[N]_{16}$	• Caracteristica(nombre : Text) → • Caracteristica(nombre : Text, props : N, pR : N, pW : N, t : N) → • escribirDatos(datos : Text) → bytesEscritos : N • notificarDatos(datos : Text) → bytesNotificados : N • activar() →

CLASE: ServicioEnEmisora

Atributos privados	Métodos públicos / contratos
• uuidServicio : $[N]_{16}$ • lasCaracteristicas : [Caracteristica]	• ServicioEnEmisora(nombreServicio : Text) → • escribeUUID() ← • anyadirCaracteristica(car : Caracteristica) → • activarServicio() →

CLASE: EmisoraBLE

Atributos privados	Métodos públicos / contratos
• nombreEmisora : Text • fabricanteID : N • txPower : Z	• EmisoraBLE(nombre : Text, fabID : N, txPwr : Z) → • encenderEmisora() → • detenerAnuncio() → • estaAnunciando() ← activo : B • emitirAnuncioIBeacon(uuid : $[N]_{16}$, major : Z, minor : Z, rssi : N) → • emitirAnuncioIBeaconLibre(carga : Text, tamanyo : N) → • anyadirServicio(servicio : ServicioEnEmisora) → ok : B

CLASE: Publicador

Atributos privados	Métodos públicos / contratos
• beaconUUID : $[N]_{16}$ • laEmisora : EmisoraBLE • RSSI : Z	• Publicador() → • encenderEmisora() → • publicarCO2(vCO2 : Z, cont : N, tEspera : Z) → • publicarTemperatura(vTemp : Z, cont : N, tEspera : Z) → • getEmisora() ← laEmisora : EmisoraBLE

1.4 Subsistema Gateway y cliente móvil — Android / Java

CLASE: TramaIBeacon

Atributos privados	Métodos públicos / contratos
• losBytes : [Z] • prefijo : $[Z]_{9}$ • advFlags : $[Z]_{3}$ • advHeader : $[Z]_{2}$ • companyID : $[Z]_{2}$ • iBeaconType : Z • iBeaconLength : Z • uuid : $[Z]_{16}$ • major : $[Z]_{2}$ • minor : $[Z]_{2}$ • txPower : Z • valida : B	• TramaIBeacon(bytes : [Z]) → • esValida() ← B • getPrefijo() ← $[Z]_{9}$ • getUUID() ← $[Z]_{16}$ • getMajor() ← $[Z]_{2}$ • getMajorInt() ← Z • getMinor() ← $[Z]_{2}$ • getMinorInt() ← Z • getTxPower() ← Z

CLASE: Utilidades

Atributos privados	Métodos públicos / contratos
• Clase pura de funciones estáticas; sin estado de instancia	• stringToBytes(texto : Text) → [Z] • stringToUUID(uuidTexto : Text) → UUID • uuidToString(uuid : UUID) → Text • uuidToHexString(uuid : UUID) → Text • bytesToString(bytes : [Z]) → Text • dosLongToBytes(msb : Z, lsb : Z) → $[Z]_{16}$ • bytesToInt(bytes : [Z]) → Z • bytesToLong(bytes : [Z]) → Z • bytesToIntOK(bytes : [Z]) → Z • bytesToHexString(bytes : [Z]) → Text

CLASE: PeticionarioREST

Atributos privados	Métodos públicos / contratos
• elMetodo : Text • urlDestino : Text • elCuerpo : Text • laRespuesta : RespuestaREST • codigoRespuesta : Z • cuerpoRespuesta : Text	• hacerPeticionREST(metodo : Text, url : Text, c : Text, cb : Resp) → • doInBackground() ← exito : B • onPostExecute(comoFue : B) →

CLASE: ServicioEscuharBeacons

Atributos privados	Métodos públicos / contratos
• tiempoDeEspera : Z • seguir : B	• onHandleIntent(intent : Intent) → • parar() → • onDestroy() →

CLASE: MainActivity (Android Coordinator)

Atributos privados	Métodos públicos / contratos
• elEscanner : BluetoothLeScanner • callbackDelEscaneo : ScanCallback	• buscarTodosLosDispositivosBTLE() → • mostrarInfoDispositivoBTLE(resultado : ScanResult) ← • buscarEsteDispositivoBTLE(dispositivoBuscado : Text) → • detenerBusquedaDispositivos() → • comprobarPermisosEscaneo() ← concedidos : B • inicializarBlueTooth() →

1.5 Subsistema Servicios Web REST y lógica — PHP / REST / UX JS

CLASE: LogicaNegocio — funciones puras de dominio

Atributos privados	Métodos públicos / contratos
• Estado agnóstico de protocolo y desacoplado de sesión de transporte	• diHola(usuario : Text) → (nombre : Text, saludo : Text) • hacerLogin(nombre : Text, password : Text) → auth : B

CLASE: ControladoresREST — endpoints HTTP

Atributos privados	Métodos públicos / contratos
• Estado: sesión PHP activa en $_SESSION["usuario"]	• rest_diHola(usuario_sesion : Text) → RespuestaSaludo • rest_hacerLogin(nombre : Text, password : Text) → RespuestaLogin

CLASE: UX_LogicaCliente — JavaScript Frontend

Atributos privados	Métodos públicos / contratos
• Comunicación asíncrona vía XMLHttpRequest / Fetch API	• diHola(cb : (err : Text, res : RespuestaSaludo)) → • hacerLogin(nombre : Text, pass : Text, cb : (res : RespuestaLogin)) →

FASE 2. COMENTARIOS Y DOCUMENTACIÓN DE CÓDIGO

Se ha auditado e inyectado documentación de arquitectura y responsabilidad en los propios archivos fuente del proyecto. Los bloques clave son:

Android — Utilidades.java

`PBIO-2025-Sprint0/PBIO-2025-Sprint0/Android-BTLE/Utilidades.java`

- Explicación de la representación binaria en complemento a 2 y justificación de la máscara 0x80 para determinar el signo negativo en enteros de longitud dinámica (1 a 4 bytes).
- Firma formal matemática en la cabecera de cada función: entradas → nombre_funcion() → salidas.
- Codificación UTF-8 incondicional en sustitución de llamadas dependientes de la plataforma.

Android — TramaIBeacon.java

`PBIO-2025-Sprint0/PBIO-2025-Sprint0/Android-BTLE/TramaIBeacon.java`

- Desglose formal de la estructura de 30 bytes del estándar Apple iBeacon: prefijo de 9 bytes y payload de 21 bytes.
- Control de precondición: longitud mínima requerida de 30 bytes para prevenir excepciones en tiempo de ejecución.

Android — MainActivity.java

`PBIO-2025-Sprint0/PBIO-2025-Sprint0/Android-BTLE/MainActivity.java`

- Descripción del flujo de escaneo BLE y su correspondencia con el modelo asíncrono de eventos ScanCallback.
- Explicación de la transición entre permisos tradicionales de Location y Android 12+ (API 31/33: BLUETOOTH_SCAN y BLUETOOTH_CONNECT).

Android — PeticionarioREST.java

`PeticionarioREST/PeticionarioREST.java y PruebaRest1-2022/.../PeticionarioREST.java`

- Contrato asíncrono RespuestaREST.
- Gestión dual de flujos: InputStream para respuestas HTTP 2xx/3xx y ErrorStream para 4xx/5xx.

Firmware — Publicador.h, EmisoraBLE.h, ServicioEnEmisora.h

`HolaMundoIBeacon/ y Arduino-NoCambia/`

- Inyección de magnitudes en payload iBeacon.
- Control del ciclo de trabajo de radio: anuncio, retardo y detención para preservación de batería.
- Saneamiento e inicialización de búferes mediante memset para evitar fuga de datos no inicializados al invertir identificadores UUID.

Backend — hacerLogin.php y diHola.php

`EsqueletoWebAppEnPHPConSesion/EsqueletoWebAppEnPHPConSesion/`

- Separación formal de capas: lógica pura de negocio frente a transporte/controlador HTTP.
- Documentación de códigos HTTP 200, 401 y 400 y de la cabecera MIME application/json; charset=utf-8.

FASE 3. AUDITORÍA DE ERRORES Y VULNERABILIDADES

La siguiente matriz consolida los defectos identificados en la inspección estática descrita en la versión v3.

Severidad	Defecto / vulnerabilidad	Impacto
CRÍTICO	Utilidades.bytesToIntOK(): máscara 0x8 en vez de 0x80; además, cast a byte en retorno y en asignación.	Interpretación de signo incorrecta y truncamiento de ent
CRÍTICO	TramaIBeacon: acceso a posiciones 0..29 sin validar longitud.	ArrayIndexOutOfBoundsException ante paquetes BLE no i
MEDIO	bytesToHexString(): añade ':' residual al final.	Formato no compatible con comparaciones estándar de c
MEDIO	getBytes()/new String() dependen de codificación de plataforma.	Posible corrupción de caracteres no ASCII.
BAJO	throw new Error(...).	Semántica inadecuada; se propone IllegalArgumentExcep
CRÍTICO	PeticionarioREST: Content-Type con 'charset-utf-8'.	Sintaxis MIME incorrecta para servidores estrictos.
CRÍTICO	DataOutputStream.writeBytes(this.elCuerpo).	Pérdida de bytes altos de caracteres Unicode.
ALTO	No se consume ErrorStream en HTTP 4xx/5xx.	Pérdida del diagnóstico proporcionado por el servidor.
ALTO	disconnect() no garantizado ante excepciones tempranas.	Posible fuga de recursos/sockets.
MEDIO	AsyncTask obsoleto desde Android 11 / API 30.	Modelo asíncrono legado con riesgos de ciclo de vida.
CRÍTICO	MainActivity.buscarEsteDispositivoBTLE(): doble startScan().	SCAN_FAILED_ALREADY_STARTED y fallo del escaneo.
ALTO	onRequestPermissionsResult(): solo inspecciona grantResults[0].	La app puede asumir permisos no concedidos.
ALTO	getBluetoothLeScanner() inmediatamente después de enable().	Condición de carrera y posible NullPointerException.
CRÍTICO	Firmware: stringAUint8AlReves() no limpia el resto del búfer.	Exposición de bytes residuales de memoria.
ALTO	PuertoSerie.esperarDisponible(): while(!Serial) sin límite.	Bloqueo indefinido en funcionamiento autónomo.
MEDIO	Publicador::laEmisora expuesto públicamente.	Ruptura de encapsulación y del ciclo de vida del publicad
CRÍTICO	Login mediante GET con nombre y password en Query String.	Exposición en historial, proxies y logs HTTP.
ALTO	Doble session_start() en rest/hacerLogin.php.	Warnings que pueden contaminar la salida JSON.
MEDIO	Ausencia de Content-Type application/json.	Contrato MIME ambiguo para consumidores REST.
MEDIO	Campo error con tipo entero en éxito y cadena en fallo.	Violación de un contrato de tipos consistente.

Detalle técnico de los hallazgos por módulo

1. Utilidades y parseo binario

- El defecto de bytesToIntOK() afecta directamente a la interpretación de telemetrías transportadas por Major/Minor.
- El bit de signo correcto para una representación de byte es 0x80. La extensión de signo debe conservar el ancho lógico del valor, no truncarlo a byte.
- TramaIBeacon debe validar como precondición que el buffer tenga al menos 30 bytes antes de realizar cualquier acceso por índice.
- El saneamiento de UTF-8 y de formatos hexadecimales reduce diferencias de representación entre emisor y receptor.

2. Comunicación REST HTTP

- El cliente debe tratar de forma distinta los cuerpos asociados a respuestas satisfactorias y de error.
- El payload JSON debe serializarse como bytes UTF-8 explícitos.
- El cierre de recursos debe quedar garantizado mediante finally o una construcción equivalente.
- AsyncTask debe considerarse una pieza de legado en una modernización Android.

3. Bluetooth y ciclo de vida

- El escaneo debe iniciarse una sola vez por ciclo lógico de búsqueda.
- Los permisos deben evaluarse individualmente según la API y operación requerida.
- El acceso al BluetoothLeScanner debe ocurrir cuando el adaptador esté realmente disponible.

4. Firmware

- Los búferes deben inicializarse antes de escribir una representación invertida de longitud fija.
- El puerto serie USB nativo no debe ser una dependencia bloqueante del funcionamiento autónomo.
- El objeto emisora debe permanecer encapsulado dentro del publicador.

5. Backend y autenticación

- Las credenciales deben viajar en POST y no formar parte de la URL.
- La sesión debe inicializarse de forma idempotente.
- Los endpoints JSON deben declarar explícitamente su MIME.
- Los campos de respuesta deben conservar tipos estables en éxito y error.

FASE 4. MEJORA Y REFACTORIZACIÓN

Las soluciones se formalizan como contratos de entrada/salida y se acompañan de fragmentos representativos del código refactorizado.

4.1 Firmas matemáticas formales

Conversión numérica con extensión de signo

$$
\begin{aligned}
&\text{bytesToIntOK} : [\ Z\ ]_n \to Z, \quad 1 \le n \le 4 \\
&\\
&\text{bytesToIntOK}(B) = \begin{cases} \sum_{i=0}^{n-1} B[i] \cdot 2^{(8(n-1-i))} & \text{si } (B[0] \wedge 128) = 0 \\ \left(\sum_{i=0}^{n-1} B[i] \cdot 2^{(8(n-1-i))} \right) - 2^{(8n)} & \text{si } (B[0] \wedge 128) \neq 0 \end{cases}
\end{aligned}
$$

Validación robusta de trama iBeacon

$$
\begin{aligned}
&\text{TramaIBeacon} : [\ Z\ ] \to (\ \text{valida} : \square, \text{Major} : \mathbb{Z}, \text{Minor} : \mathbb{Z}, \text{TxPower} : \mathbb{Z}\ ) \\
&\\
&\text{esValida}(T) = (|T| \ge 30) \wedge (T[5] = 0x4C) \wedge (T[6] = 0x00) \wedge (T[7] = 0x02)
\end{aligned}
$$

Cliente HTTP REST con detección dual de streams

$$
\begin{aligned}
&\text{peticionHTTP} : (\text{metodo} : \text{Text}, \text{url} : \text{Text}, \text{payload} : \text{Text}) \\
&\qquad \to (\text{status} : \mathbb{Z}, \text{cuerpo} : \text{Text}) \\
&\\
&\text{StreamSelector}(\text{status}) = \begin{cases} \text{InputStream} & \text{si status} < 400 \\ \text{ErrorStream} & \text{si status} \ge 400 \end{cases}
\end{aligned}
$$

Inversión segura con relleno de búfer

$$
\begin{aligned}
&\text{stringAUint8AlReves} : (\text{str} : \text{Text}, \text{tamMax} : \mathbb{N}) \to [\ \mathbb{N}\ ]_\text{tamMax} \\
&\\
&\forall j \in [0, \text{tamMax}-1], \\
&\text{buffer}[j] = \begin{cases} \text{str}[\text{tamMax}-1-j] & \text{si } (\text{tamMax}-1-j) < |\text{str}| \\ 0 & \text{en caso contrario} \end{cases}
\end{aligned}
$$

4.2 Código Android — Utilidades.java

Archivo modificado: PBIO-2025-Sprint0/PBIO-2025-Sprint0/Android-BTLE/Utilidades.java

```
public static int bytesToIntOK( byte[] bytes ) {
    if ( bytes == null || bytes.length == 0 ) return 0;
    if ( bytes.length > 4 ) {
        throw new IllegalArgumentException(
            "bytesToIntOK: longitud máxima superada (máximo 4 bytes)" );
    }
    int res = 0;
    for ( byte b : bytes ) {
        res = (res << 8) | (b & 0xFF);
    }
    // Extensión de signo matemática si el bit 7 está activo
    if ( (bytes[0] & 0x80) != 0 ) {
        int bitsFaltantes = (4 - bytes.length) * 8;
        res = (res << bitsFaltantes) >> bitsFaltantes;
    }
    return res;
}
```

- Se sustituye (bytes[0] & 0x8) por (bytes[0] & 0x80).
- Se aplica propagación aritmética mediante desplazamientos.
- Se elimina el carácter ':' residual de bytesToHexString().
- Se fuerza StandardCharsets.UTF_8 en conversiones de texto.
- Se reemplaza throw new Error por IllegalArgumentException.

4.3 Código Android — TramaIBeacon.java

```
public TramaIBeacon( byte[] bytes ) {
    if ( bytes == null || bytes.length < 30 ) {
        this.valida = false;
        return;
    }
    this.losBytes = bytes;
    this.prefijo = Arrays.copyOfRange( losBytes, 0, 9 );
```

```
        this.uuid = Arrays.copyOfRange( losBytes, 9, 25 );
        this.major = Arrays.copyOfRange( losBytes, 25, 27 );
        this.minor = Arrays.copyOfRange( losBytes, 27, 29 );
        this.txPower = losBytes[29];

        this.advFlags = Arrays.copyOfRange( prefijo, 0, 3 );
        this.advHeader = Arrays.copyOfRange( prefijo, 3, 5 );
        this.companyID = Arrays.copyOfRange( prefijo, 5, 7 );
        this.iBeaconType = prefijo[7];
        this.iBeaconLength = prefijo[8];

        this.valida = ((companyID[0] & 0xFF) == 0x4C
            && (companyID[1] & 0xFF) == 0x00
            && this.iBeaconType == 0x02);
    }
```

- Guarda defensiva ante arrays nulos o menores de 30 bytes.
- Validación del Company ID de Apple (0x004C) y subtipo de proximidad (0x02).
- Métodos getMajorInt() y getMinorInt() para entrega directa de valores enteros.

4.4 Código Android — MainActivity.java

- Eliminación del startScan() redundante en buscarEsteDispositivoBTLE().
- Verificación exhaustiva de todos los permisos relevantes en grantResults.
- Comprobación preventiva de nulidad de BluetoothLeScanner.

4.5 Código Android — PeticionarioREST.java

```
// Selección dinámica de flujo según código de estado HTTP
InputStream is = ( this.codigoRespuesta < HttpURLConnection.HTTP_BAD_REQUEST )
        ? connection.getInputStream()
        : connection.getErrorStream();
```

- Content-Type corregido a application/json; charset=utf-8.
- Payload escrito mediante getBytes(StandardCharsets.UTF_8).
- Lectura de getErrorStream() para códigos HTTP ≥ 400.
- Cierre incondicional de sockets en finally.

4.6 Firmware IoT — ServicioEnEmisora.h, PuertoSerie.h, Publicador.h

```
memset( pUint, 0, tamMax );  // saneamiento previo del búfer
```

- ServicioEnEmisora.h: memset antes de invertir la cadena.
- PuertoSerie.h: timeout de 2000 ms en esperarDisponible().
- Publicador.h: encapsulación de laEmisora y acceso mediante getEmisora().

Inyección de magnitudes en iBeacon

```
Major = (MedicionesID << 8) + contador
Minor = valorMedicion
```

El esquema reserva el byte alto de Major para identificar el tipo de medición y el byte bajo para el contador; Minor transporta el valor de la medición.

4.7 Backend Web — rest/hacerLogin.php y rest/diHola.php

- Eliminación de la segunda llamada redundante a session_start().
- Control idempotente mediante session_status() === PHP_SESSION_NONE.
- Cabecera Content-Type: application/json; charset=utf-8.
- Códigos de respuesta semánticos: 200 OK, 401 Unauthorized y 400 Bad Request.
- Tipado consistente del campo error en la respuesta JSON.

Cierre técnico

El diseño consolidado separa explícitamente tres dominios: adquisición/publicación en el nodo IoT, descubrimiento y transporte en el gateway móvil, y lógica/servicios en backend. Las refactorizaciones descritas corrigen defectos de representación binaria, validación de tramas, ciclo de vida BLE, codificación HTTP/UTF-8, gestión de recursos, saneamiento de memoria y exposición de credenciales, manteniendo como

objetivo un contrato lógico estable entre capas.